

# SWP OpenMP: Task 3 Kmeans

## Our CUDA Approach

Group 1: Felix Krückel, Luke Hassel, Gerrik Berner

RWTH Aachen

February 1, 2026

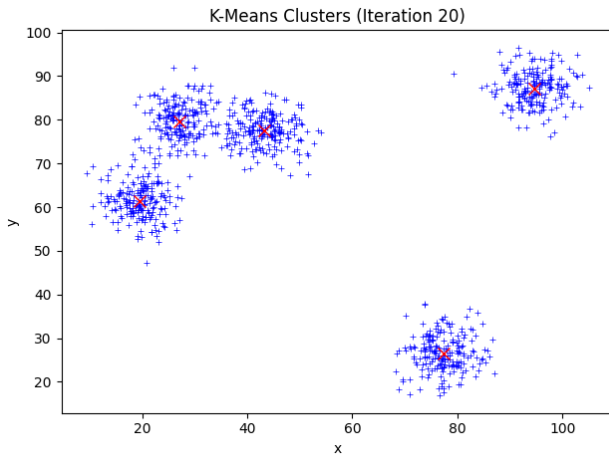
# Recap: K-Means Algorithm

- 1 **Initialization:** Select  $k$  initial centroids (randomly).
- 2 **Assignment:** Assign each data point to the nearest centroid.
- 3 **Update:** Recalculate centroids as the mean of assigned points.
- 4 **Repeat:** Until convergence or max iterations.

## Pseudocode:

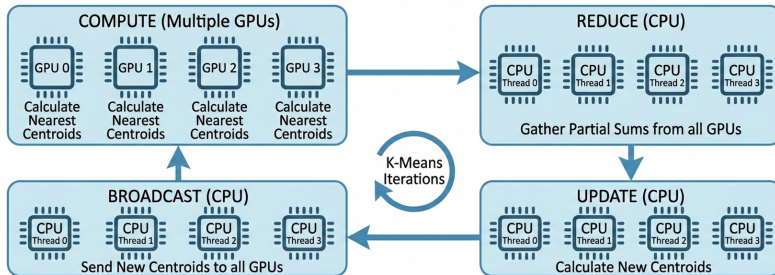
```
Initialize centroids = random(k)
While not converged:
  For point in data:
    closest = argmin(distance(point, c))
    assign(point, closest)
  For c in centroids:
    c = mean(assigned_points)
```

# Example



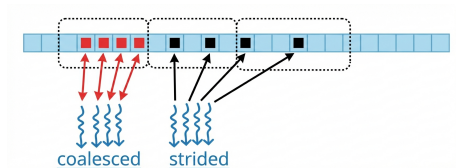
Visualization of final cluster assignments.

# Computation Flow





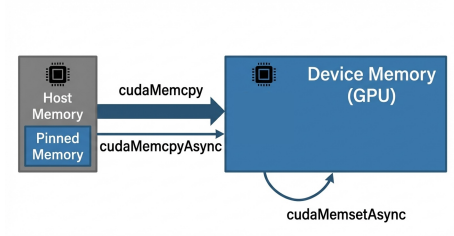
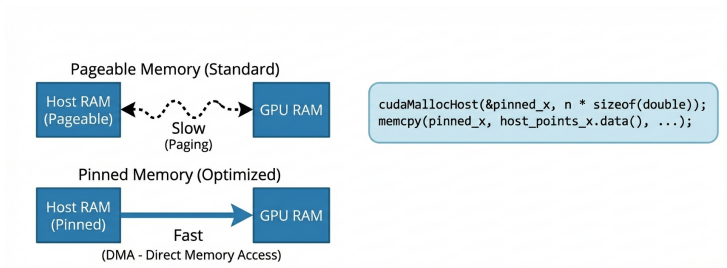
# Memory Utilisation



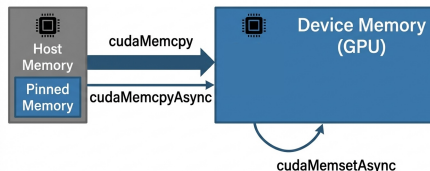
Store the points and centroids in a coalesced manner:

$[x_0, y_0, x_1, y_1, x_2, y_2, \dots] \rightarrow [x_0, x_1, x_2, \dots] \text{ AND } [y_0, y_1, y_2, \dots]$

# Memory Utilisation

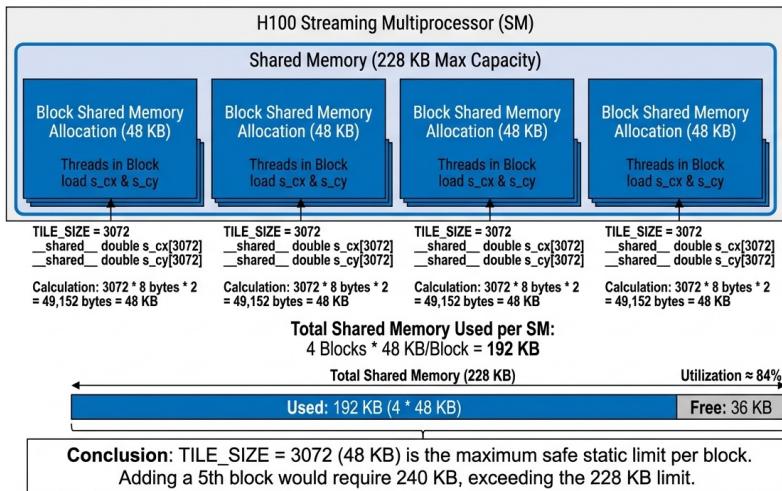


# Memory Utilisation

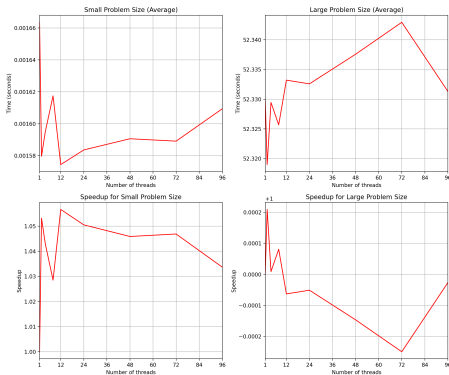


- `cudaMalloc(ptr, size)`: Allocates memory on GPU device
- `cudaMallocHost(ptr, size)`: Allocates **pinned memory** on host
- `cudaMemcpy(dst, src, size, dir)`: Synchronous copy between host ↔ device
- `cudaMemsetAsync(ptr, val, count, stream)`: Asynchronously fills device memory with a constant byte value

# Cache Blocking



# Performance Plots



# Approximation

Instead of using the full dataset in every iteration, we use a random subset of it.

